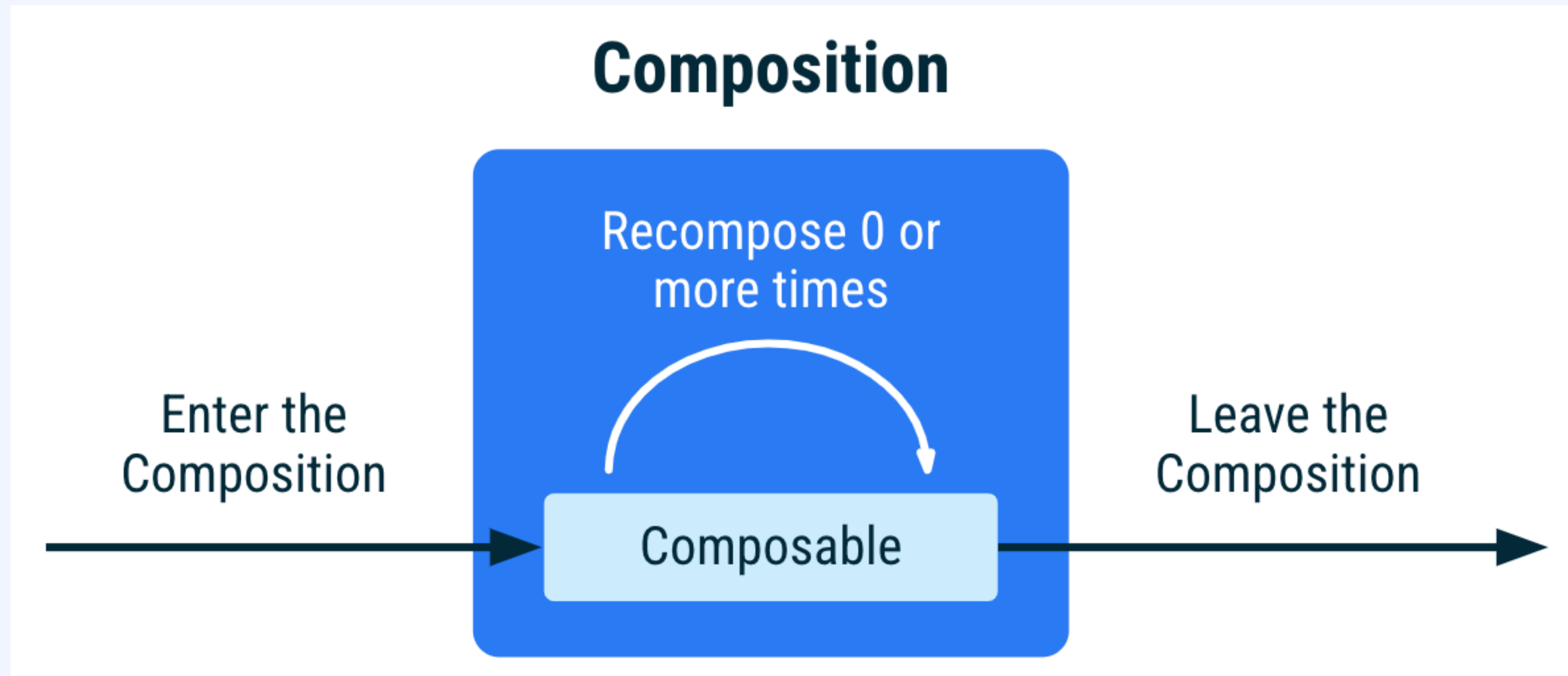


Compose 활용

2 Recomposition

Recomposition

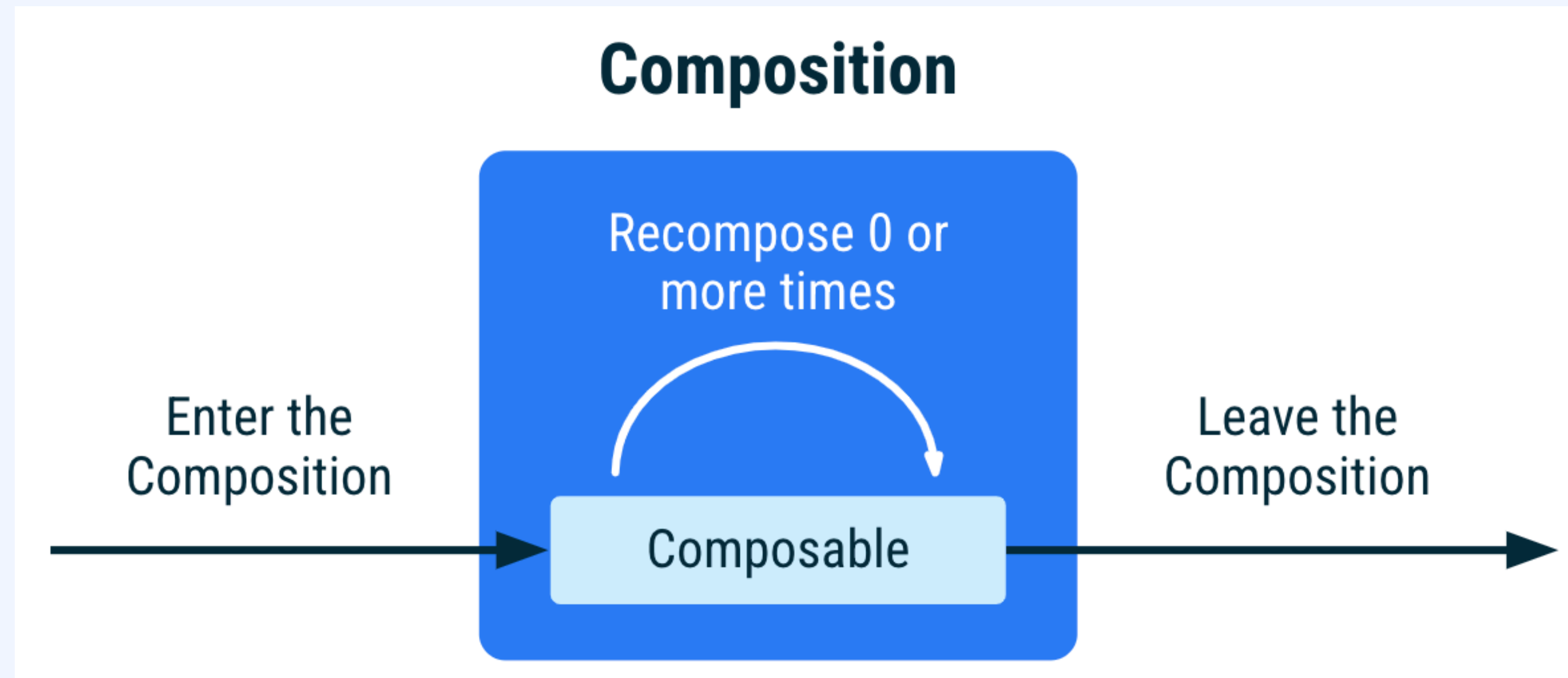
2. Recomposition



컴포지션 - 앱의 UI를 설명 컴포저블을 호출해 생성. UI를 기술하는
컴포저블의 트리 구조.

Recomposition

2. Recomposition



초기 컴포지션 - UI를 그리기 위해 호출한 콤포저블을 추적

리컴포지션 - 상태가 바뀌었을 때 예약되며 (0회 이상) 변경점을 반영하려 컴포지션을 업데이트 함. (수정하는 유일한 방법)

Recomposition

2. Recomposition

State<T>가 변경되면 리컴포지션이 트리거 됨.

State<T>를 읽는 컴포저블과, 여기에서 호출되는 컴포저블이 대상이 됨.

모든 입력이 안정적이고 변경되지 않았으면 건너뛸 수 있음.

리컴포지션 스킵

2. Recomposition

모든 입력이 안정적이고 변경되지 않았으면 건너뛸 수 있음.

안정적인 타입은 아래를 지켜야 함.

- 두 인스턴스 equals 결과가 영원히 같은 경우.
- 공개 프로퍼티가 변경되면 콤포지션에 알려야 함.
- 모든 공개 프로퍼티는 안정적이어야 함.

@Stable이 표기되지 않아도 Compose 컴파일러가 안정적이라고
간주하는 공통 타입들

- 모든 프리미티브 타입: Boolean, Int, Long, Float 등
- 문자열
- 모든 함수 타입 (람다)

안정적이라고 추론 할 수 없는 경우 @Stable을 표기해야 함

MutableStat는 안정적

2. Recomposition

MutableState는 안정적.

State의 value 프로퍼티 값 변경이 알려지기 때문.

@Stable 표시

```
// Marking the type as stable to favor  
skipping and smart recompositions.  
@Stable  
interface UiState<T : Result<T>> {  
    val value: T?  
    val exception: Throwable?  
  
    val hasError: Boolean  
        get() = exception != null  
}
```

안정적으로 다루기 위해 @Stable을 표기

같은 컴포지션 여러번 호출

2. Recomposition

```
@Composable
fun MyComposable() {
    Column {
        Text("Hello")
        Text("World")
    }
}
```

MyComposable

Column

Text

Text

컴포지션이 여러번 호출되면 여러 인스턴스가 배치

상태에 따라 콤포저블 호출이 달라지는 경우

2. Recomposition

```
@Composable
fun LoginScreen(showError: Boolean) {
    if (showError) {
        LoginError()
    }
    LoginInput() // This call site affects where LoginInput is placed in
Composition
}

@Composable
```

LoginScreen

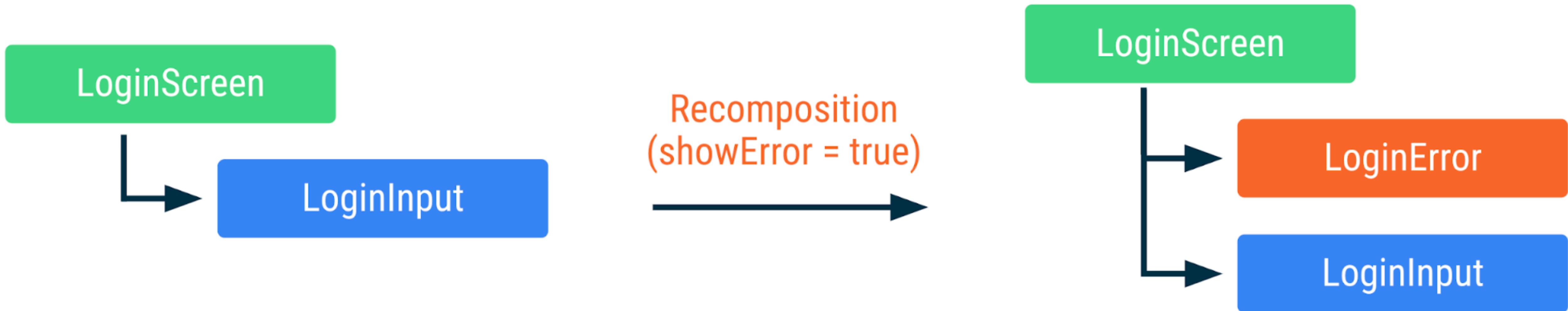
LoginInput

Recomposition
(showError = true)

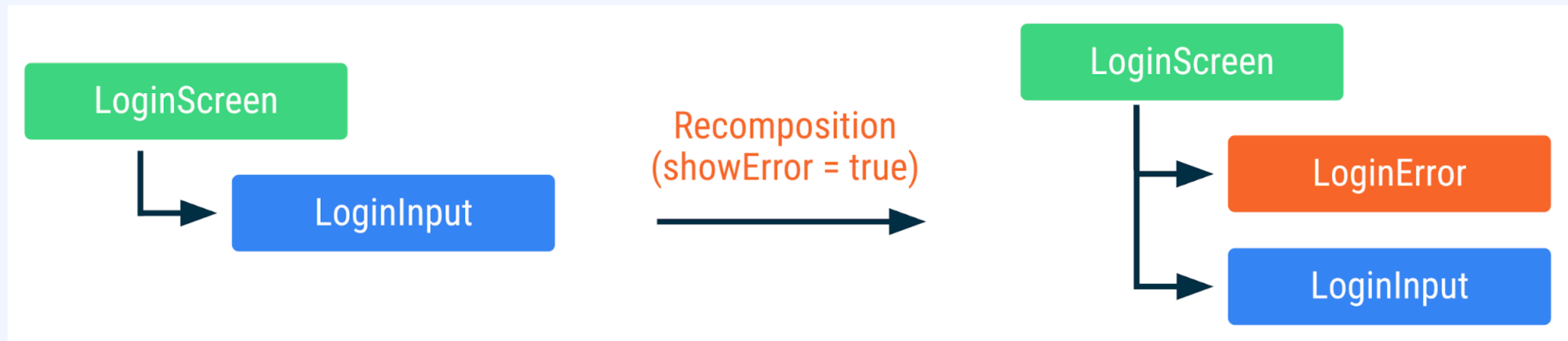
LoginScreen

LoginError

LoginInput



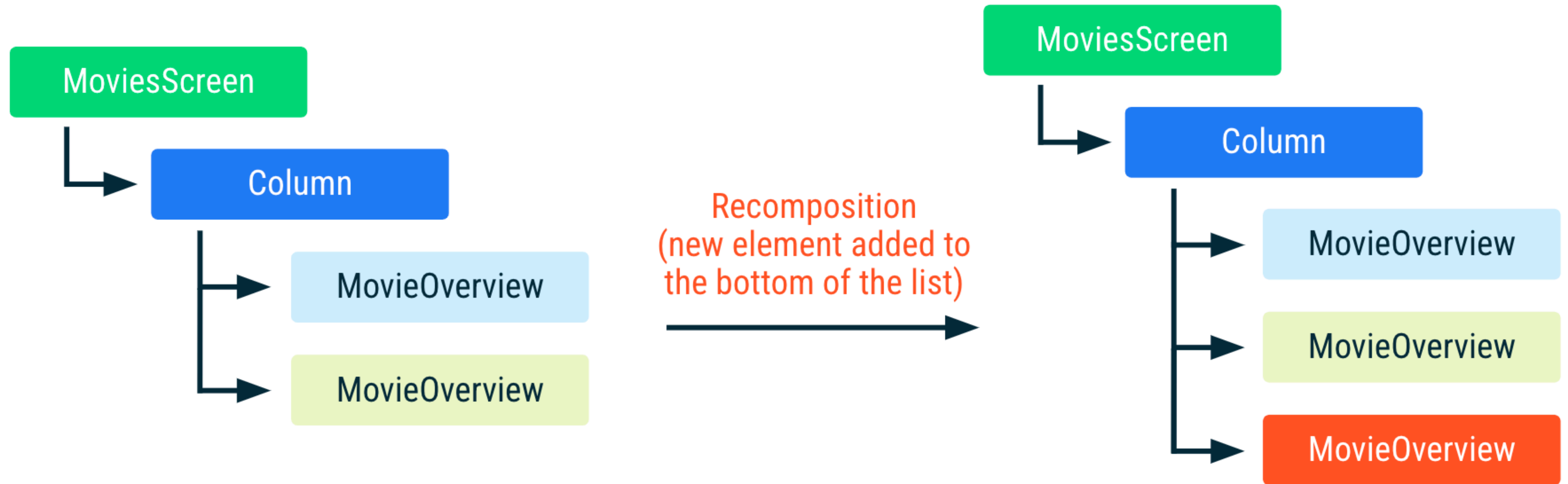
상태에 따라 컴포저블 호출이 달라지는 경우



리컴포지션에서 다른 컴포지션을 호출하는 경우 컴포저블들이 호출되는지 여부를 체크해 두 컴포지션에서 모두 호출되고 입력이 바뀌지 않은 컴포저블은 재구성하지 않음.

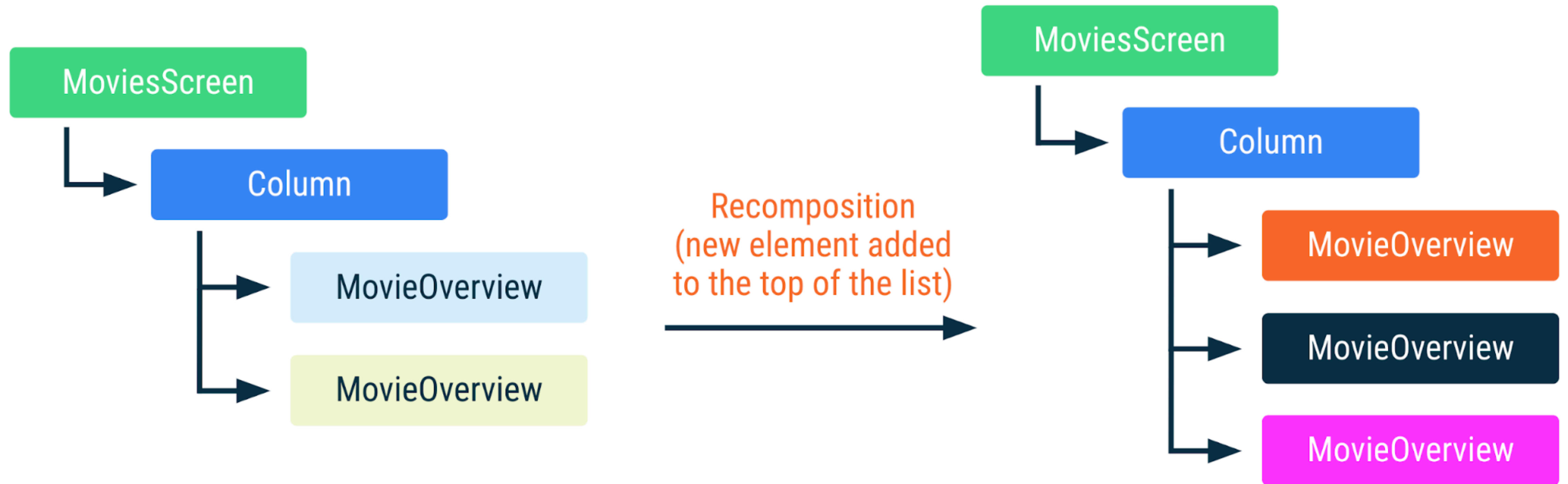
리컴포지션의 행복한 경우

2. Recomposition



리컴포지션의 불행한 경우

2. Recomposition



스마트 리컴포지션

2.
Recomposition

```
@Composable
fun MoviesScreen(movies: List<Movie>) {
    Column {
        for (movie in movies) {
            key(movie.id) { // Unique ID for
this movie
                MovieOverview(movie)
            }
        }
    }
}
```

key를 지정해서 재사용에 도움을 주기

Key 컴포저블 지원 내장 컴포저블

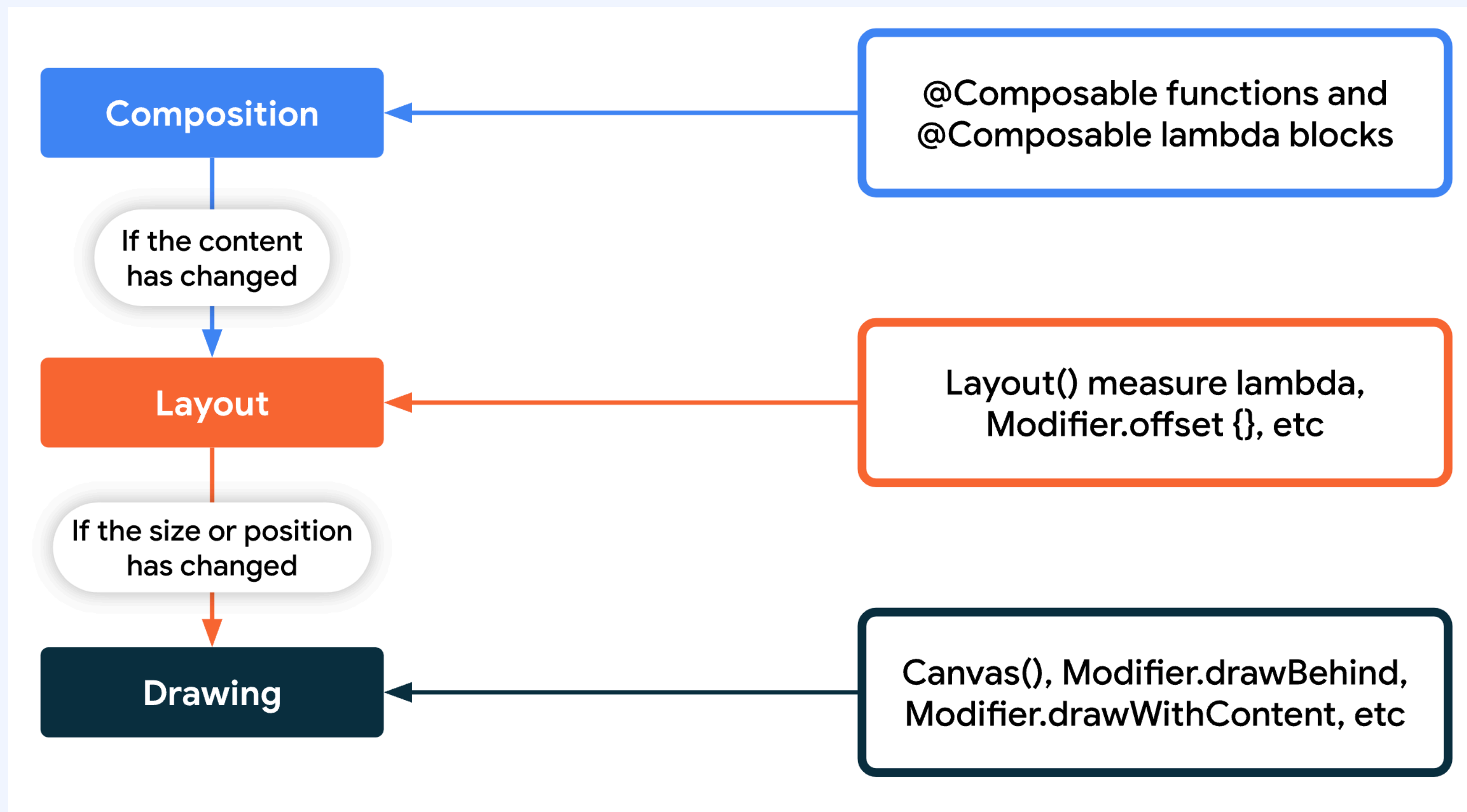
2.
Recomposition

```
@Composable
fun MoviesScreen(movies: List<Movie>) {
    LazyColumn {
        items(movies, key = { movie ->
movie.id }) { movie ->
            MovieOverview(movie)
        }
    }
}
```

LazyColumn은 key 컴퍼저블을 지원합니다.

렌더링 단계

2. Recomposition



렌더링 단계

2. Recomposition

